Quick Setup Guide for CardinalTalent

This guide will help you get CardinalTalent running on your local machine in just a few minutes.

Prerequisites Checklist

- [ ] Node.js v18+ installed (Download (https://nodejs.org/))
- [ ] PostgreSQL v14+ installed (Download (https://www.postgresql.org/download/))
- [ ] Git installed
- [ ] Code editor (VS Code recommended)

Step-by-Step Setup

1. Get the Code

```
git clone https://github.com/cardinalhire/ctnew.git
cd ctnew
git checkout sk
```

2. Install Dependencies

```
npm install
```

This will install all required packages for both frontend and backend.

3. Set Up PostgreSQL

Option A: Using psql (Command Line)

```
# Start PostgreSQL (varies by OS)
# Windows: PostgreSQL should start automatically
# macOS: brew services start postgresql
# Linux: sudo systemctl start postgresql

# Connect to PostgreSQL
psql -U postgres

# Create database
CREATE DATABASE cardinaltalent;

# Exit psql
\q
```

Option B: Using pgAdmin (GUI)

1. Open pgAdmin
2. Connect to your PostgreSQL server
3. Right-click “Databases” → “Create” → “Database”
4. Name it `cardinaltalent`
5. Click “Save”

4. Configure Environment

Copy the example environment file:

```
cp .env.example .env
```

Then edit `.env` and update your PostgreSQL password:

```
DB_PASSWORD=your_actual_postgres_password
```

Important: Replace `your_actual_postgres_password` with your PostgreSQL password!

5. Initialize Database

Run the migration to create tables and seed data:

```
npm run migrate
```

You should see:

```
✅ Database migration completed successfully!

Default admin user created:
  Email: admin@cardinaltalent.com
  Password: admin123
  ⚠️  PLEASE CHANGE THE DEFAULT PASSWORD IN PRODUCTION!
```

6. Start the Application

```
npm run dev
```

This starts both:
- **Backend API**: http://localhost:3001
- **Frontend**: http://localhost:5173

7. Access the Application

Open your browser and go to: **http://localhost:5173**

Login with:
- **Email**: `admin@cardinaltalent.com`
- **Password**: `admin123`

Verification Steps

Check if PostgreSQL is Running

Windows:

```
pg_isready
```

macOS:

```
brew services list | grep postgresql
```

Linux:

```
sudo systemctl status postgresql
```

Check if Ports are Available

Check port 3001 (Backend):

```
# Windows
netstat -ano | findstr :3001

# macOS/Linux
lsof -i :3001
```

Check port 5173 (Frontend):

```
# Windows
netstat -ano | findstr :5173

# macOS/Linux
lsof -i :5173
```

Test the API

Once the server is running, test the health endpoint:

```
curl http://localhost:3001/api/health
```

You should see:

```
{
  "status": "ok",
  "timestamp": "2026-02-09T...",
  "uptime": 1.234
}
```

Common Issues & Solutions

Issue: “Database connection failed”

Solution:
1. Check if PostgreSQL is running
2. Verify database credentials in `.env`
3. Ensure database `cardinaltalent` exists
4. Check if PostgreSQL is listening on port 5432

Issue: “Port 3001 already in use”

Solution:
1. Find and kill the process using port 3001:

```bash
# Windows
netstat -ano | findstr :3001
taskkill /PID /F

# macOS/Linux
lsof -ti:3001 | xargs kill -9
```

2. Or change the PORT in .env to a different number (e.g., 3002)

Issue: “Cannot find module…”

Solution:

```
rm -rf node_modules package-lock.json
npm install
```

Issue: Migration fails with “relation already exists”

Solution:
Drop and recreate the database:

```
-- In psql or pgAdmin
DROP DATABASE IF EXISTS cardinaltalent;
CREATE DATABASE cardinaltalent;
```

Then run migration again:

```
npm run migrate
```

Issue: “ECONNREFUSED” when starting the app

Solution:
1. Make sure you ran `npm run migrate` first
2. Check if PostgreSQL is running
3. Verify `.env` has correct database credentials

Next Steps

Once the application is running:

1. **Login as Admin**
   - Email: `admin@cardinaltalent.com`
   - Password: `admin123`
   - **Change the password immediately!**

2. **Create Test Accounts**
   - Create accounts for different roles (Talent, Employer, Recruiter)
   - Test the different dashboards

3. **Test Resume Upload**
   - Login as a Talent user

   - Navigate to Settings or Dashboard
   - Upload a test resume (PDF, DOC, or DOCX)

4. **Explore Features**
   - Browse the different dashboards
   - Test the authentication flow
   - Try password reset functionality

Development Workflow

Starting Development

```
# Start everything
npm run dev

# Or start separately
npm run server:dev  # Backend only
npm run client:dev  # Frontend only
```

Making Changes

1. **Frontend changes**: Vite will hot-reload automatically
2. **Backend changes**: tsx watch will restart the server automatically
3. **Database changes**: Update `schema.sql` and create migration scripts

Testing Changes

```
npm test              # Run tests once
npm run test:watch    # Run tests in watch mode
```

Building for Production

```
npm run build
```

This creates:
- `dist/client/` - Frontend production build
- `dist/server/` - Backend production build

Project Structure Overview

```
ctnew/
├── server/           # Backend (Express + PostgreSQL)
│   ├── database/     # DB connection & schema
│   ├── routes/       # API endpoints
│   ├── services/     # Business logic
│   └── middleware/   # Auth middleware
│
├── src/              # Frontend (React + TypeScript)
│   ├── components/   # Reusable UI components
│   ├── pages/        # Page components
│   ├── lib/          # Utilities (API client, auth)
│   └── hooks/        # Custom React hooks
│
├── uploads/          # Local file storage (auto-created)
└── .env              # Environment config (not committed)
```

Available Scripts

- `npm run dev` - Start both frontend and backend in development mode
- `npm run client:dev` - Start frontend only
- `npm run server:dev` - Start backend only
- `npm run build` - Build for production
- `npm run migrate` - Run database migrations
- `npm test` - Run tests
- `npm run lint` - Lint code

Need Help?

If you encounter any issues not covered here:

1. Check the full README.md (./README.md) for detailed information
2. Review the error messages carefully
3. Check if PostgreSQL is running and accessible
4. Verify all environment variables in `.env`
5. Try restarting the application
6. Check the console logs for both frontend and backend

Security Reminders

⚠️ **Before deploying to production**:

- [ ] Change the default admin password
- [ ] Generate a strong JWT_SECRET
- [ ] Configure proper SMTP for emails
- [ ] Set up SSL/TLS certificates
- [ ] Review and update CORS settings
- [ ] Enable proper logging and monitoring
- [ ] Set up database backups

- [ ] Remove or disable development endpoints

Happy coding! 🚀